Synthetic Gaussian Datasets

Much of the code here will follow from the notebook in the repository, however, this pdf will include exercises and open questions meant to guide your learning of the flow matching process.

1. Setup

```
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt
import sys
import os
sys.path.insert(0, os.path.abspath('.'))

from gaussian_data  import get_gaussian_dataset, N, DIM, evaluate,  _build_sigma
from gaussian_model import MLP_Residual, cfm_loss, sample, sincos_embed, SIGMA_MIN

device = "cuda" if torch.cuda.is_available() else "cpu"
```

2. Data

With the hopes of visualization in mind, we aim to have our data be able to be represented as a 32 by 32 grayscale image, hence we let the data be $x \in \mathbb{R}^{1024}$ which is a flattened vector of $x \in \mathbb{R}^{32\times 32}$. To see some structure arise, we choose to create a Gaussian which has a block covariance matrix. Mathematically, we have that we want to start from $x_0 \sim \mathcal{N}(0, Id)$ which we want to generate $x_1 \sim \mathcal{N}(0, \Sigma)$ where $\Sigma \in \mathbb{R}^{1024\times 1024}$. For this experiment, we arbitrarily choose that we want four different blocks in our covariance matrix which then takes the form

$$\Sigma = (1-\rho)I_{1024\times 1024} + \rho \begin{pmatrix} \mathbf{1}_{256\times 256} & \mathbf{0}_{256\times 256} & \mathbf{0}_{256\times 256} & \mathbf{0}_{256\times 256} \\ \mathbf{0}_{256\times 256} & \mathbf{1}_{256\times 256} & \mathbf{0}_{256\times 256} & \mathbf{0}_{256\times 256} \\ \mathbf{0}_{256\times 256} & \mathbf{0}_{256\times 256} & \mathbf{1}_{256\times 256} & \mathbf{0}_{256\times 256} \\ \mathbf{0}_{256\times 256} & \mathbf{0}_{256\times 256} & \mathbf{0}_{256\times 256} & \mathbf{1}_{256\times 256} \end{pmatrix}$$

where $\rho < 1$ is the intensity, (higher ρ leads to more intense checkerboard pattern). We arbitrarily use $\rho = 0.85$ for our generated data.

```
block_size = 4
rho = 0.85
Sigma = _build_sigma(N, block_size = block_size, rho = rho)

data = get_gaussian_dataset(Sigma = Sigma, n_samples=10000)
data = torch.from_numpy(data).to(device)
```

To visualize the data we can run the following command.

```
def show_sample(x: torch.Tensor):
    img = x.cpu().numpy().reshape(32, 32)
    plt.imshow(img, cmap='gray', interpolation='nearest')
    plt.axis('off')
    plt.show()

show_sample(data[0])
```

3. Forward Process

We follow the formulation of conditional flow matching introduced by Lipman et al. Let $\rho : [0,1] \times \mathbb{R}^d \to \mathbb{R}_{>0}$ be a probability density path such that

$$\int_{\mathbb{R}^d} \rho_t(x)\, dx = 1 \quad \text{for all } t \in [0,1].$$

Our goal is to learn a time-dependent vector field $v : [0,1] \times \mathbb{R}^d \to \mathbb{R}^d$ which generates a flow ϕ_t satisfying

$$\frac{d}{dt}\phi_t(x) = v_t(\phi_t(x)),$$

and such that the induced densities evolve according to

$$\rho_t = [\phi_t]_\star \rho_0.$$

Given a probability path $\rho_t(x)$ with associated generating vector field $u_t(x)$, a natural objective is

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}(0,1),\, x\sim\rho_t} \left\| v_t(x;\theta) - u_t(x) \right\|^2.$$

We choose ρ_0 as a simple base distribution and $\rho_1 \approx q$, the target distribution.

Remark 3.1. Directly sampling from ρ_t is typically intractable, which motivates the conditional formulation below.

Reference. Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, *Flow Matching for Generative Modeling*, 2022.

3.1. Conditional Flow Matching

We introduce conditional probability paths of the form

$$\rho_t(x \mid x_1) = \mathcal{N}\big(x \mid \mu_t(x_1),\, \sigma_t(x_1)^2 I\big).$$

We impose boundary conditions

$$\mu_0(x_1) = 0, \quad \sigma_0(x_1) = 1, \qquad \mu_1(x_1) = x_1, \quad \sigma_1(x_1) = \sigma_{\min},$$

so that $\rho_0(x) = \mathcal{N}(0, I)$ and $\rho_1(x \mid x_1)$ concentrates at x_1.

The conditional flow matching objective is

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}(0,1),\, x_1\sim q,\, x\sim\rho_t(\cdot|x_1)} \left\| v_t(x;\theta) - u_t(x \mid x_1) \right\|^2,$$

which has the same optimal vector field as $\mathcal{L}_{\mathrm{FM}}$.

3.2. Optimal Transport Path

We consider the optimal transport conditional path defined by

$$\mu_t(x_1) = t x_1, \qquad \sigma_t(x_1) = 1 - (1 - \sigma_{\min})t.$$

This induces the interpolation

$$\psi_t(x_0) = \big(1 - (1 - \sigma_{\min})t\big)x_0 + t x_1,$$

and the corresponding vector field

$$u_t(x \mid x_1) = \frac{x_1 - (1 - \sigma_{\min})x_0}{1 - (1 - \sigma_{\min})t}.$$

Under this path, the conditional flow matching loss can be written as

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}(0,1),\, x_1\sim q,\, x_0\sim\rho_0} \left\| v_t(\psi_t(x_0);\theta) - \big(x_1 - (1 - \sigma_{\min})x_0\big) \right\|^2,$$

where $\sigma_{\min} = 10^{-4}$ throughout.

Remark 3.2. This formulation avoids sampling from ρ_t directly by instead sampling (x_0, x_1) and evaluating along the interpolation $\psi_t(x_0)$.

Remark 3.3. The resulting objective provides an unbiased estimator of the original flow matching objective while being computationally tractable.

Exercise 1 (Conditional Flow Matching Loss Implementation) *Consider the function* `cfm_loss` *defined in* `gaussian_model.py`*. Your goal is to implement the conditional flow matching (CFM) loss corresponding to the optimal transport path described above.*

You are given a batch of samples $x_1 \in \mathbb{R}^{B\times d}$ drawn from the target distribution $q(x)$, and a model $v_t(x;\theta)$ implemented as `model(x, t)`*.*

Complete the implementation of `cfm_loss` *by performing the following steps:*

1. *Sample $x_0 \sim \mathcal{N}(0, I)$ with the same shape as x_1.*
2. *Sample $t \sim \mathcal{U}(0, 1)$ independently for each data point in the batch.*
3. *Construct the interpolation*
$$\psi_t(x_0) = \big(1 - (1 - \sigma_{\min})t\big)x_0 + tx_1.$$
4. *Compute the target velocity*
$$u_t = x_1 - (1 - \sigma_{\min})x_0.$$
5. *Evaluate the model at (x_t, t) and return the mean squared error*
$$\frac{1}{B}\sum_{i=1}^{B}\left\|v_t(x_t^{(i)};\theta) - u_t^{(i)}\right\|^2.$$

Your final implementation should return a scalar loss suitable for backpropagation.

4. Model

We use a residual MLP $v_\theta : \mathbb{R}^{1024} \times [0, 1] \to \mathbb{R}^{1024}$ to learn the vector field. At each hidden layer, the timestep t is injected via a learned projection of the sinusoidal time embedding, allowing each layer to respond to the noise level independently:

$$h \leftarrow h + \sigma\big(W_\ell h + P_\ell e_t\big)$$

```
model = MLP_Residual(
    input_size     = DIM,    # 1024
    hidden_size    = 512,
    amount_layers  = 4,
    output_size    = DIM,    # 1024
    time_dimension = 64,
).to(device)
```

5. Training

We minimize the OT-CFM loss using Adam. At each gradient step we sample a random minibatch from the dataset, draw fresh base samples $x_0 \sim \mathcal{N}(0, I)$ and timesteps $t \sim \mathcal{U}(0, 1)$ inside `cfm_loss`, and update the model weights.

Exercise 2 (Training Loop) *In* `gaussian_model.py`*, you implemented the function* `cfm_loss`*. You will now use this loss to train the model.*

Write the training loop so that it performs the following steps for a fixed number of gradient updates:

1. *Sample a random minibatch of target points x_1 from the dataset.*

2. *Compute the conditional flow matching loss by calling* `cfm_loss(model, x1)`.
3. *Clear the old gradients stored in the optimizer.*
4. *Backpropagate the loss.*
5. *Take one optimizer step.*
6. *Record the scalar loss value after each gradient step.*
7. *Update the progress bar so that it displays the current loss.*

After training, plot the recorded loss values as a function of gradient step.

Your implementation should use Adam with learning rate `LR`*, run for* `N_STEPS` *iterations, and use mini-batches of size* `BATCH_SIZE`.

6. Sampling

Once trained, we generate samples by integrating the learned vector field $v_\theta(x_t, t)$ forward from $t = 0$ to $t = 1$ using Euler's method:

$$x_{t+\Delta t} = x_t + v_\theta(x_t, t) \cdot \Delta t, \qquad \Delta t = \frac{1}{T}$$

starting from $x_0 \sim \mathcal{N}(0, I)$. After T steps we arrive at $x_1 \approx \mathcal{N}(0, \Sigma)$.

Exercise 3 (Sampling by Euler Integration) *After training the vector field, we would like to generate samples by solving the learned flow ODE forward in time.*

Implement the function `sample` *in* `gaussian_model.py` *so that it performs Euler integration starting from base samples $x_0 \sim \mathcal{N}(0, I)$ and evolves them from time $t = 0$ to time $t = 1$ using the learned vector field.*

Your implementation should do the following:

1. *Put the model in evaluation mode.*
2. *Draw* `n_samples` *initial points from $\mathcal{N}(0, I)$ in $\mathbb{R}^{\mathrm{DIM}}$.*
3. *Set the Euler step size to $\Delta t = 1/$*`n_steps`.
4. *For each step $i = 0, 1, \ldots,$* `n_steps` $- 1$*, form the time vector*

$$t_i = \frac{i}{\texttt{n_steps}}$$

   for all samples in the batch.
5. *Update the particles using the Euler rule*

$$x \leftarrow x + v_t(x)\,\Delta t.$$

6. *Return the final batch of samples.*

Your final function should return a tensor of shape `[n_samples, DIM]`.